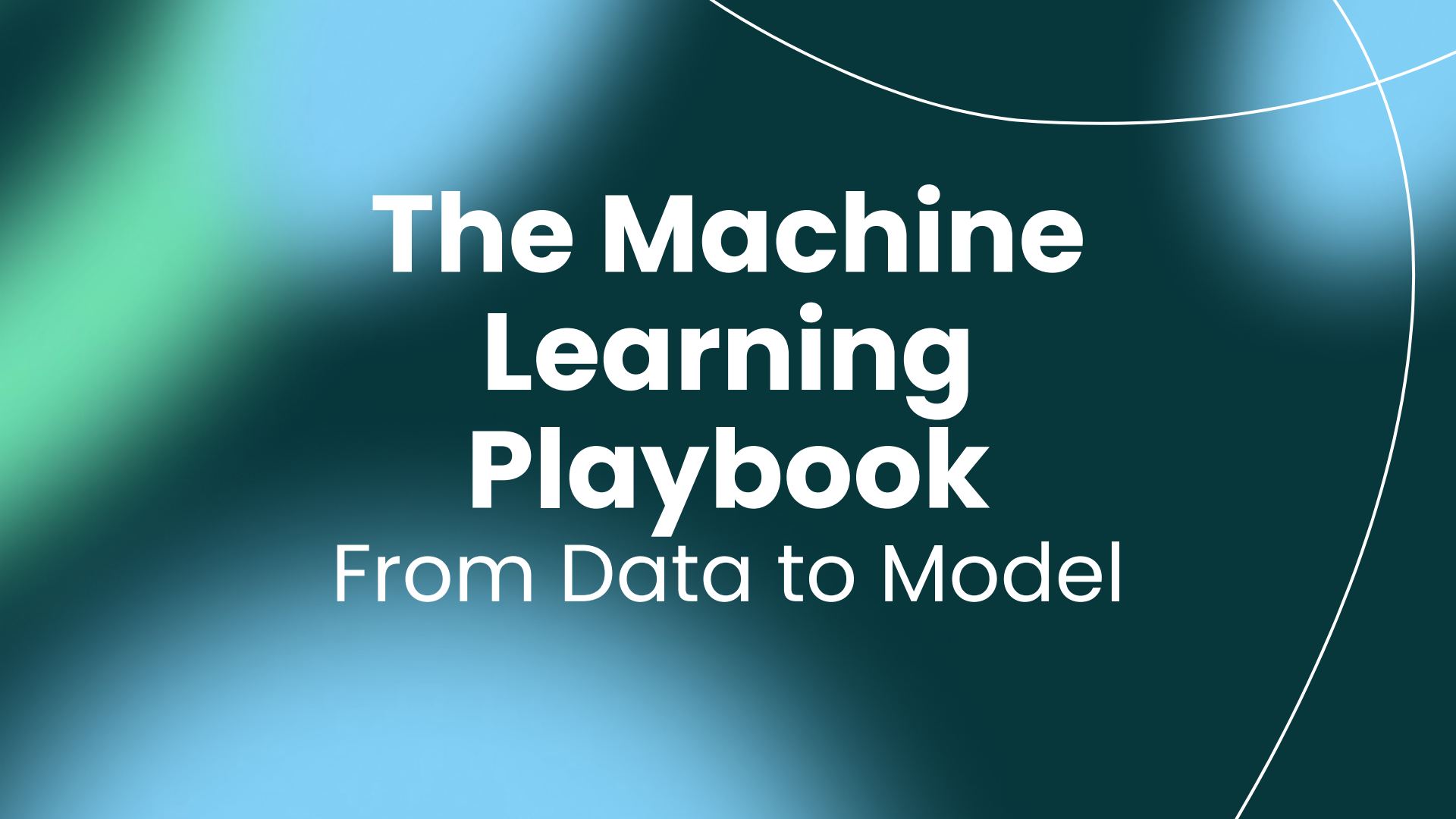
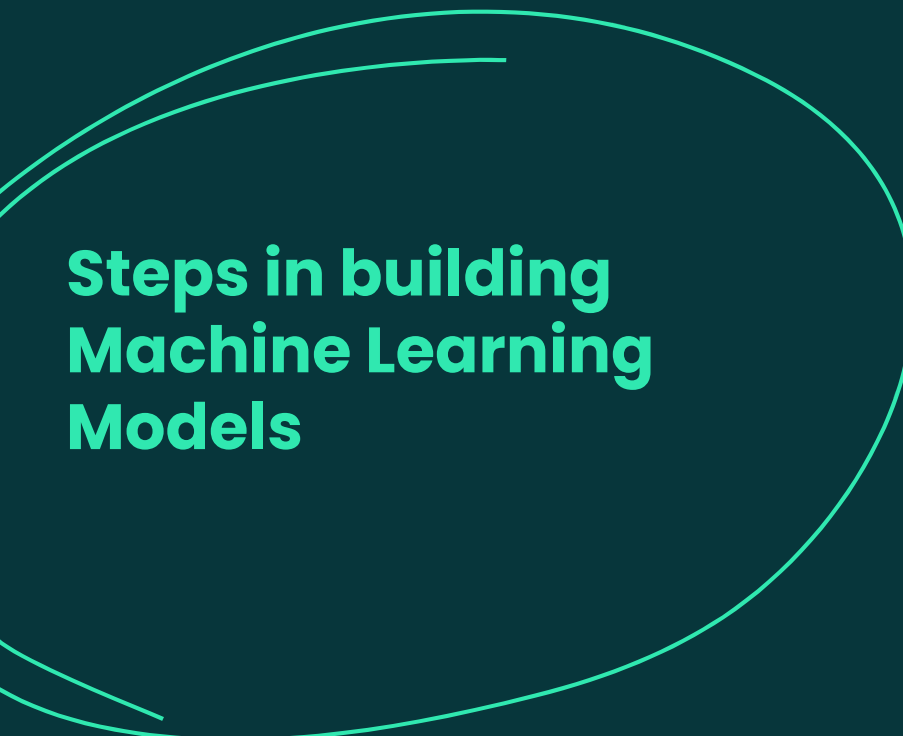




# **The Machine Learning Playbook**

From Data to Model



## **Steps in building Machine Learning Models**

1. EDA (Exploratory Data Analysis)
2. Data Cleaning
3. Encoding Categorical Variables
4. Feature Selection and Feature Engineering
5. Sampling and Splitting Data
6. Model Selection
7. Selecting Loss Function
8. Selecting Evaluation Metrics
9. Tuning Hyperparameters and Stopping Criteria
10. Improving Robustness with Cross-validation

# **1. Exploratory Data Analysis (EDA)**

# Data Exploration

- Summary stats: mean, median, std, percentiles
- Distribution plots (histograms, Kernel Density Estimation - KDE)
- Correlations
  - Pearson: Continuous data
  - Spearman: Ranks
- Check class imbalance (for classification), where positive class is rare. Examples: detecting disease, predicting defaults



# Data Exploration

## Understand feature importance

- Univariate analysis (single feature)
- Bivariate analysis (feature vs target)
- Multivariate (correlation matrix, PCA)

## Importance of EDA by model types

- Linear models: To check assumptions
- Tree models: Less sensitive but still useful
- Neural networks: For feature scaling insights

## **2. Data Cleaning**

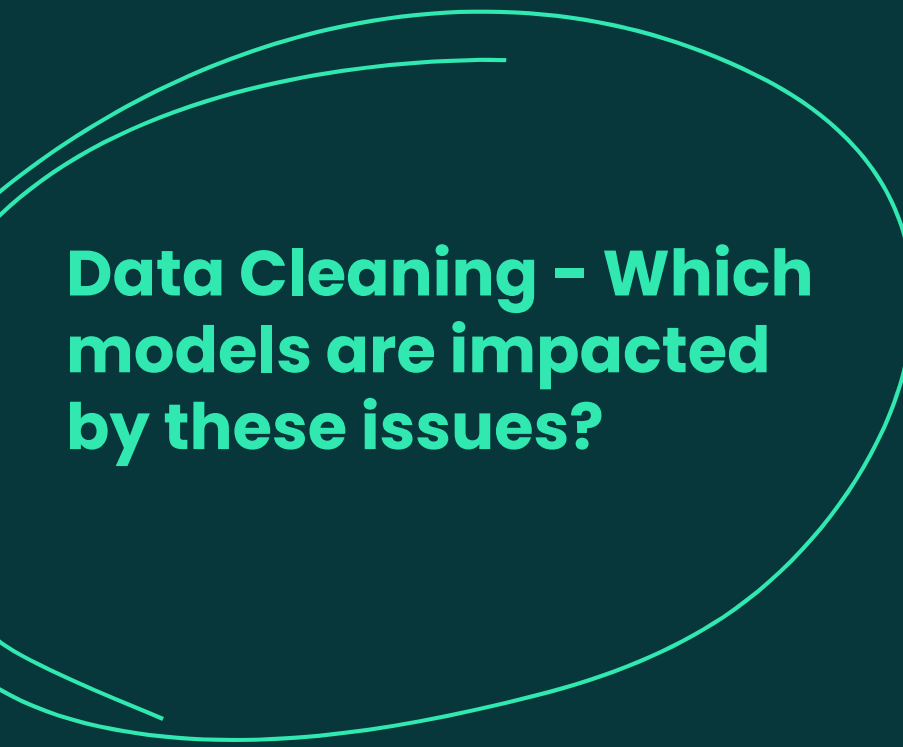
# Data Cleaning – Common Issues and Fixes

## (a) Missing Data

- Detect: `.isnull()`, heatmaps
- What is pattern of missing data? (important for strategy) : MCAR (Missing Completely at Random), MAR (Missing at Random), MNAR (Missing Not at Random)

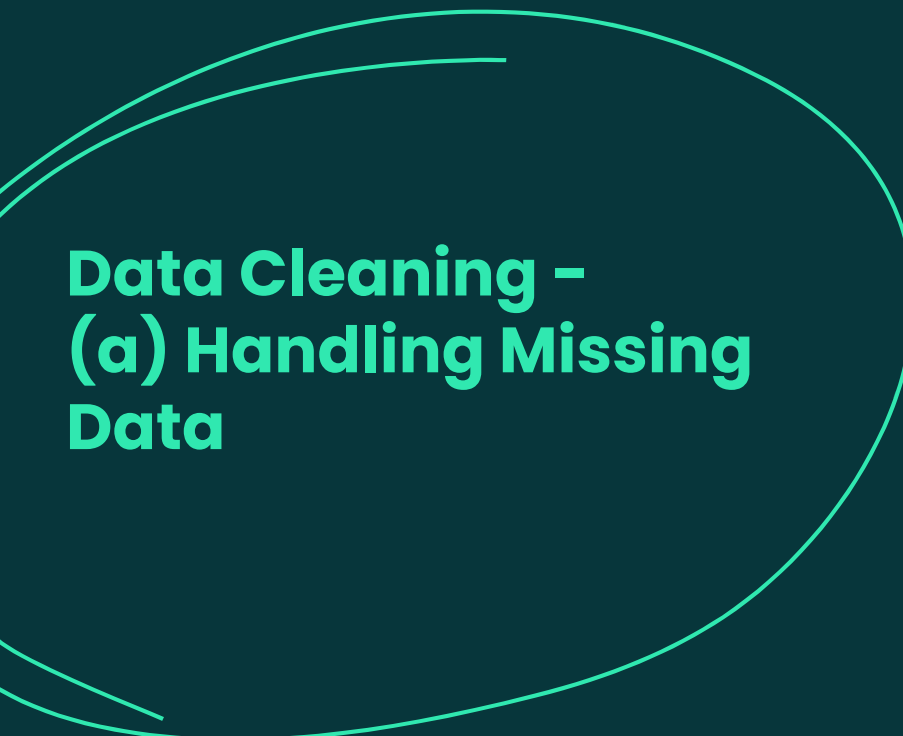
(b) Outliers Detection: IQR method, Z-score, Isolation Forest (advanced)

(c) Scaling - Needed when feature magnitudes differ



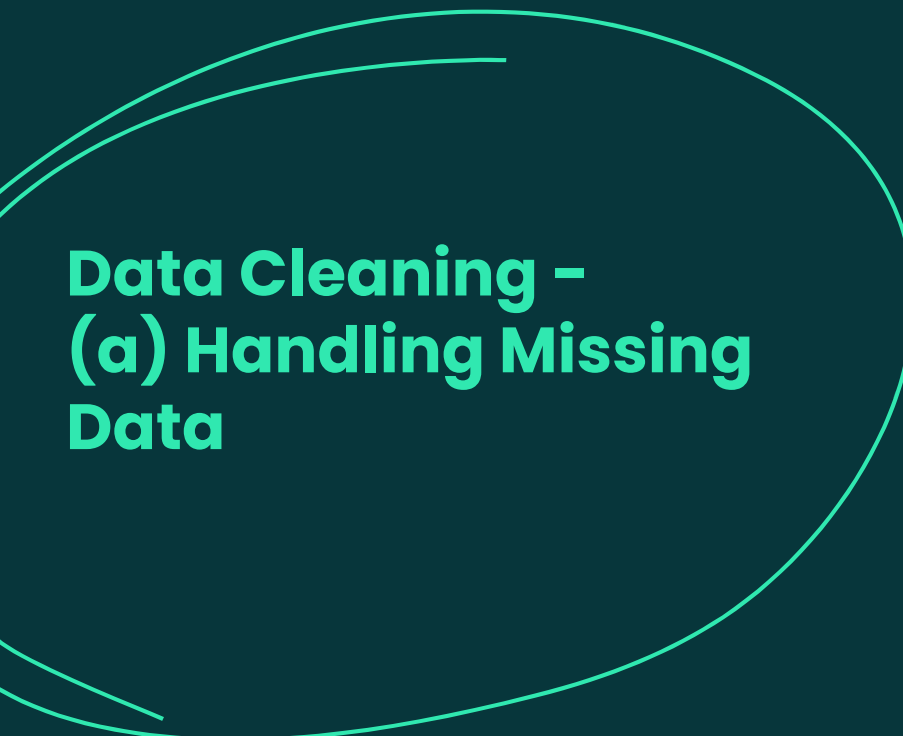
## **Data Cleaning – Which models are impacted by these issues?**

| Step                  | Models                        |
|-----------------------|-------------------------------|
| Missing data handling | ALL models                    |
| Outlier handling      | Linear, SVM, Neural Nets      |
| Scaling               | Linear, SVM, KNN, Neural Nets |



## **Data Cleaning – (a) Handling Missing Data**

1. Remove data
  - Drop rows or columns
  - Use when: Small % missing, Not critical feature
2. Simple Imputation
  - Mean / Median / Mode
  - Can result in a a bias
3. Model-based handling
  - Some models handle missing directly: XGBoost / LightGBM



## **Data Cleaning – (a) Handling Missing Data**

4. Advanced Imputation: Using models to calculate missing values

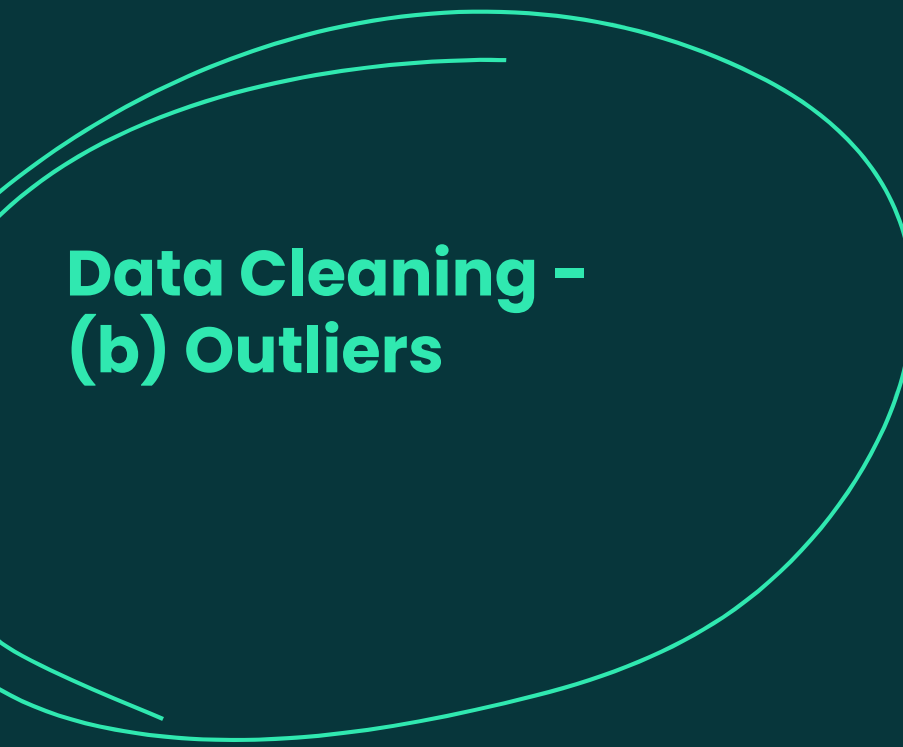
- KNN imputation
- Regression imputation
- Iterative: Multivariate Imputation by Chained Equations (MICE)

5. Add “missing indicator” as missing data can contain ‘information’

- Add binary column: was\_missing

## Data Cleaning – (a) Handling Missing Data: Model- wise Recommendations

| Approach          | Models                |
|-------------------|-----------------------|
| Mean/Median/Mode  | Linear models         |
| KNN               | Distance-based models |
| Built-in handling | Tree boosting models  |
| Indicator         | Neural nets, trees    |



## Data Cleaning – (b) Outliers

1. Trimming:

Remove outliers beyond a threshold (percentile or absolute)

2. Winsorization:

Replace outliers with minimum and maximum value of thresholds (e.g. 5% and 95%)

3. Transformation:

Log or square root transformation

## Data Cleaning - (c) Scaling

To ensure that all features contribute equally to the model

1. Standardization (Z-Score Scaling):  
Centers data so it has a mean of 0 and a standard deviation of 1.
2. Normalization (Min-Max Scaling):  
Rescales data into a fixed range, typically 0 to 1.

(Cont.)

## Data Cleaning – (c) Scaling (Cont.)

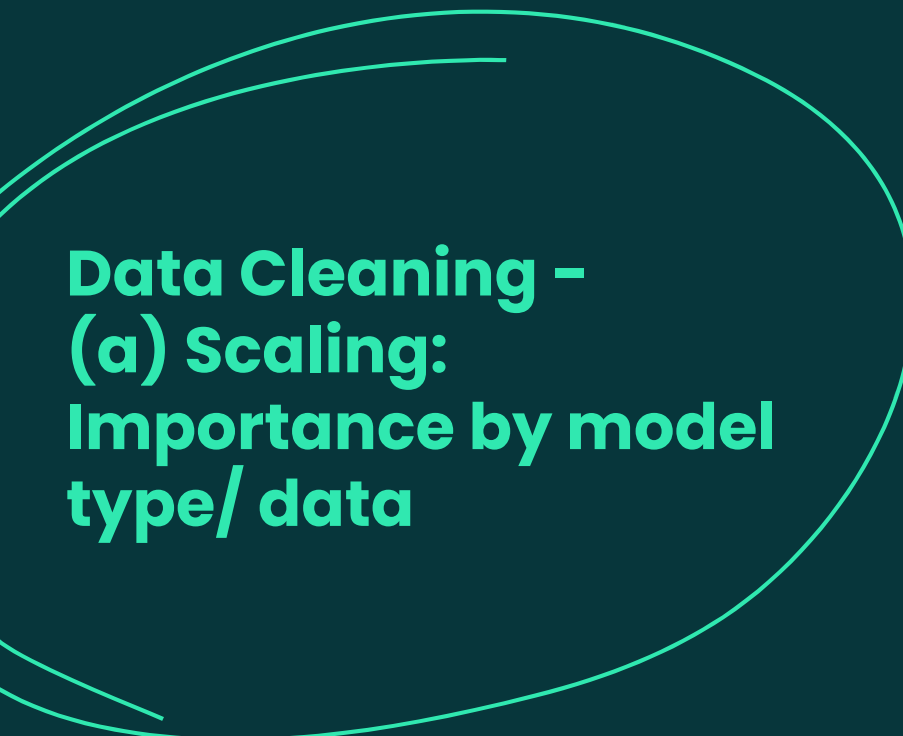
### 3. Robust Scaling:

Uses the median and Interquartile Range (IQR) to scale features

### 4. Max Absolute Scaling:

Scales data based on its maximum absolute value, resulting in a range of -1 to 1

### 5. Power & Quantile Transformation (Advanced)



## **Data Cleaning – (a) Scaling: Importance by model type/ data**

| Approach        | Models/ Data                 |
|-----------------|------------------------------|
| Standardization | Linear, SVM                  |
| Normalization   | Neural Networks,<br>KNN      |
| Robust Scaling  | With significant<br>outliers |

# **3. Encoding Categorical Variables**

## Data Cleaning – Categorical Data

1. One-Hot Encoding
  - Most common
  - Works well for: Linear models, Neural nets
2. Label Encoding
  - Assign integer values
  - Works for: Tree models (no ordinal assumption issue)

(Cont.)

## Data Cleaning - Categorical Data (Cont.)

### 3. Target Encoding

- Replace with mean target value
- Useful for high-cardinality features

### 4. Embeddings

- Learned representations
- Used in:
  - Neural networks

## Data Cleaning – Categorical Data Importance by models

| Step            | Models        |
|-----------------|---------------|
| One-hot         | Linear, NN    |
| Label encoding  | Trees         |
| Target encoding | Boosting      |
| Embeddings      | Deep learning |

# **4. Feature Selection and Feature Engineering**



# Feature Selection

1. Statistical Selection: Statistical relation between feature and target
  - a. K-Best
  - b. Variance Threshold filtering
  - c. Correlation-based filtering
2. Search based: Evaluate different subsets of features
  - a. Recursive Feature Elimination (RFE)
  - b. Forward Selection
  - c. Backward Elimination

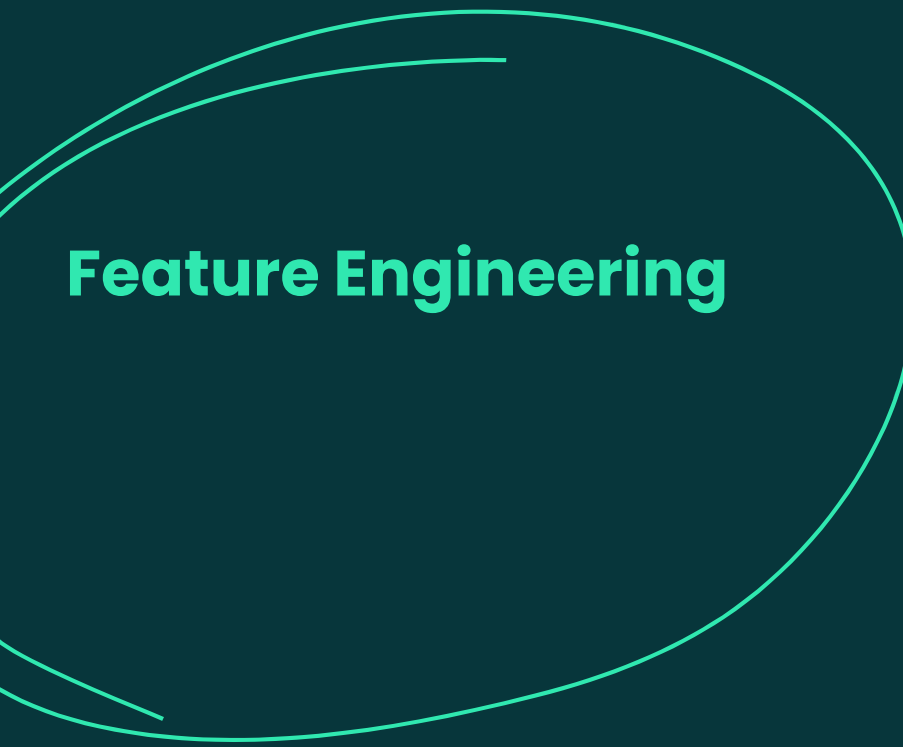


## Feature Selection

3. Embedded Methods (Built into Model): Selection in model training
  - a. Lasso (L1 Regularization)
  - b. Tree based models
  - c. Boruta
4. Dimensionality Reduction (Feature Extraction): Creates new features and reduces interpretability
  - a. PCA (Principal Component Analysis)
  - b. Autoencoders

## Feature Selection – Model- wise Recommendations

| Approach           | Models                          |
|--------------------|---------------------------------|
| K-best             | Linear, SVM, KNN                |
| Correlation filter | Linear                          |
| RFE                | Linear, SVM, Tree               |
| Lasso              | Linear/ Logistic                |
| Boruta             | Tree                            |
| PCA                | Linear, KNN,<br>Neural Networks |



# Feature Engineering

Creating new features from the existing features. Key categories include

1. Date/ time decomposition:  
Example: day\_of\_week, date, is\_weekend
2. Binning: converting continuous variable into categories. Example: age into child/ teen/ adult/ senior
3. Combination of features. Example: price per sq ft
4. Log/ Square root for skewed data
5. Polynomial features

# **5. Sampling and Splitting Data**



# Sampling Techniques

Sampling approach depends on the nature of the data

## 1. Random split

- Default

## 2. Stratified split

- For classification (preserve class ratio)

## 3. Time-based split

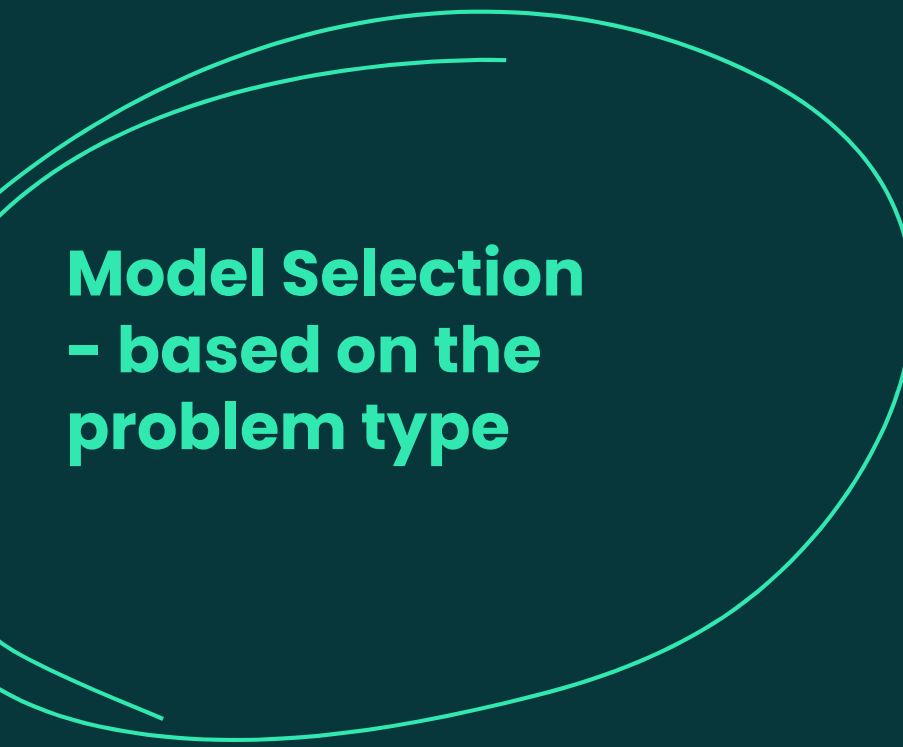
- For time series



## **Sampling Techniques – For imbalanced data**

1. Oversampling
  - a. Duplicate minority samples
  - b. SMOTE (Synthetic Minority Oversampling Technique)
  - c. ADASYN (Adaptive Synthetic Sampling)
2. Undersampling
  - a. Random Undersampling
  - b. Edited Nearest Neighbors (ENN)
  - c. Cluster Centroids
3. Algorithm-Level Methods
  - a. Class Weights
  - b. Cost-Sensitive Learning

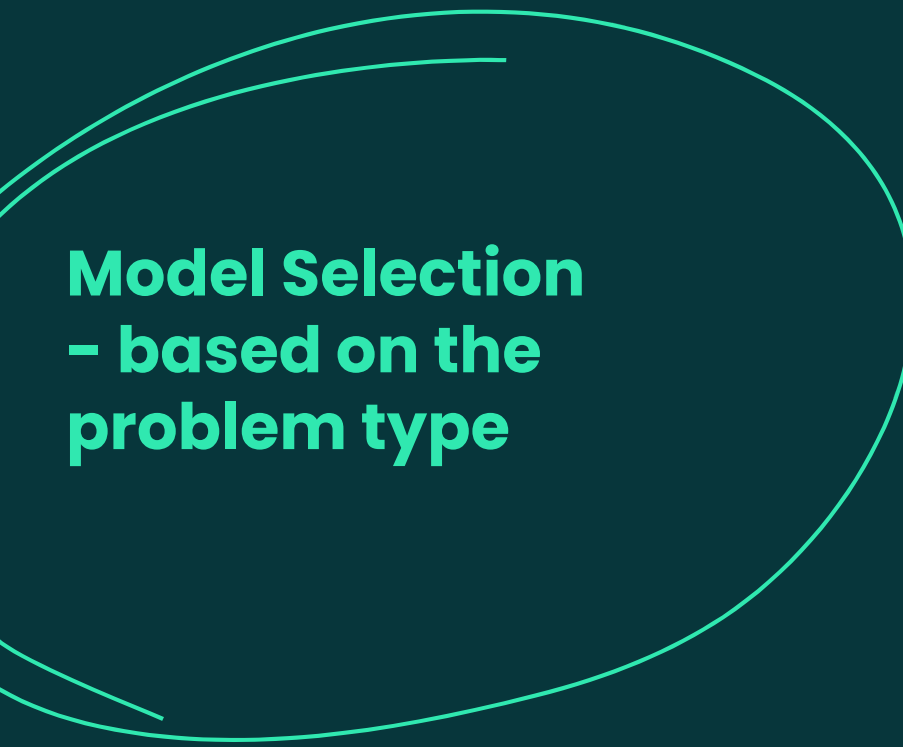
# **6. Model Selection**



## **Model Selection – based on the problem type**

### (a) Regression

- Linear Regression
- Ridge / Lasso
- Random Forest
- Gradient Boosting (XGBoost, LightGBM)
- Neural Networks



## **Model Selection – based on the problem type**

### (b) Classification

- Logistic Regression
- SVM
- Random Forest
- Gradient Boosting
- Neural Networks

### (c) Clustering

- K-Means
- DBSCAN
- Hierarchical

## Model Selection – Data Size

| Data Size         | Models                                        |
|-------------------|-----------------------------------------------|
| Small (<10K rows) | Linear models, SVM, KNN                       |
| Medium (10K–1M)   | Trees, Random Forest, Gradient Boosting       |
| Large (>1M)       | Neural Networks, Boosting, distributed models |

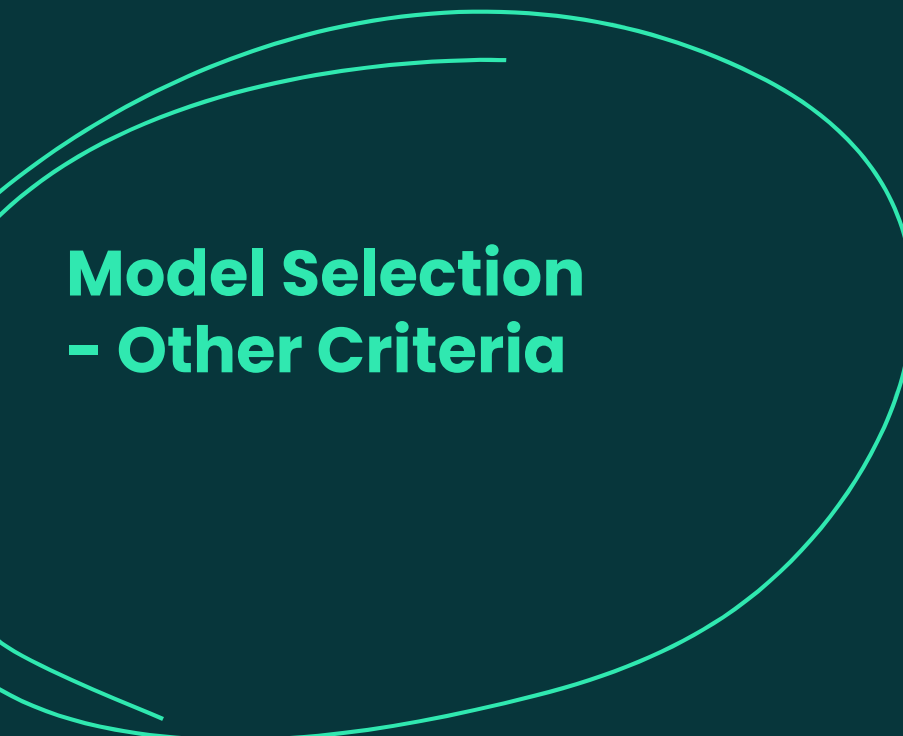


## Model Selection – Features

- Number of Features
  - High-dimensional ( $p \gg n$ ):  
Lasso, Elastic Net, SVM
- Feature Types
  - Categorical (many categories):  
Tree-based (e.g., boosting)
- Linearity
  - Non-linear: Trees, Boosting,  
Neural Nets

## Model Selection – Interpretability

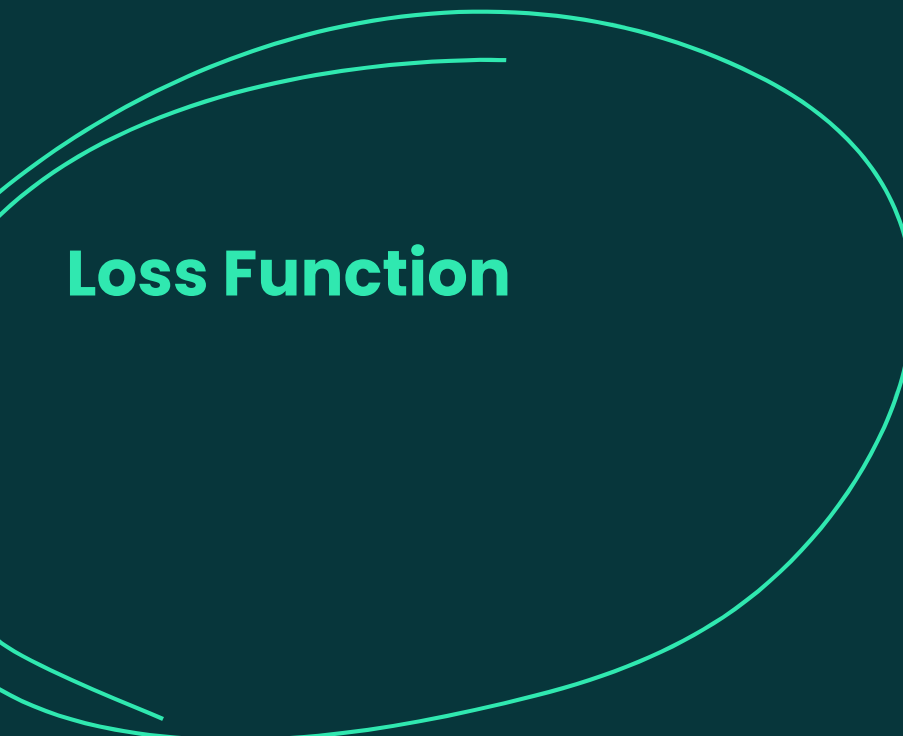
| Interpretability                 | Models                                       |
|----------------------------------|----------------------------------------------|
| High interpretability            | Linear models, Decision Trees                |
| Medium interpretability          | Random Forest, Gradient Boosting (with SHAP) |
| Low interpretability (Black Box) | Neural Networks                              |



## Model Selection – Other Criteria

- Deployment Constraints
  - Large-scale serving: Boosting, optimized NN
  - Low latency: Linear, small trees
- Handling Missing Data
  - Missing data present: Tree-based

# **7. Selecting Loss Functions**



# Loss Function

## Regression

- MSE (Mean Squared Error)
- MAE (Mean Absolute Error)
- Huber Loss

## Classification

- Binary Cross-Entropy
- Categorical Cross-Entropy
- Hinge Loss (SVM)

## Special cases

- Imbalanced data: Weighted loss, Focal loss

## Loss Function – Model-wise Recommendations

| Model               | Loss                          |
|---------------------|-------------------------------|
| Linear Regression   | MSE                           |
| Logistic Regression | Log loss                      |
| SVM                 | Hinge                         |
| Neural Nets         | Customizable                  |
| Boosting            | Custom (often logloss / RMSE) |

# **8. Selecting Evaluation Metrics**



## Evaluation Metrics

1. Regression
  - a. MAE (Mean Absolute Error)
  - b. MSE (Mean Squared Error)/  
RMSE (Root Mean Squared Error)
  - c.  $R^2$  (R-squared)
2. Probabilistic Predictions (Logistic Regression, Neural Networks)
  - a. Log Loss (Cross-Entropy)
  - b. Brier Score



## Evaluation Metrics

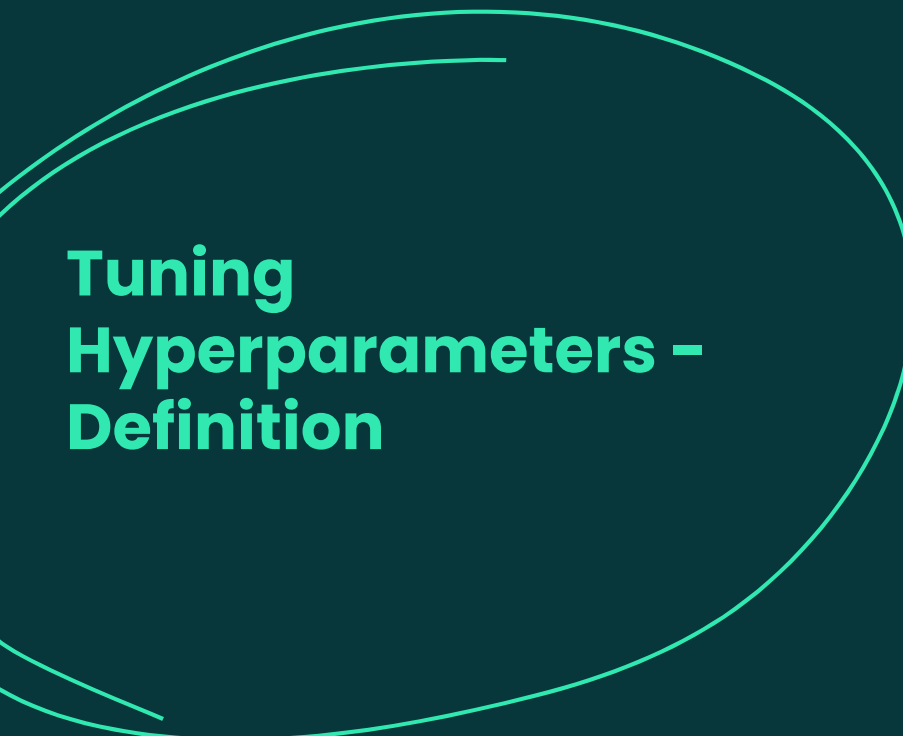
3. Classification (Balanced data)
  - a. Accuracy
  - b. ROC-AUC
4. Classification (Imbalanced data)
  - a. Precision: Of all items labeled as positive, how many were actually positive?
  - b. Recall: Of all actual positive cases, how many did we find?
5. Clustering Models
  - a. Silhouette Score
  - b. Within cluster variance



## Evaluation Metrics

6. Ranking/ Recommendation Systems
  - a. Precision@K
  - b. Recall@K
  - c. NDCG (Normalized Discounted Cumulative Gain)
  - d. MAP (Mean Average Precision)
  - e. MRR (Mean Reciprocal Rank)
7. Business specific: Classification  
Threshold based on cost of errors
  - a. False positives costly: Focus on Precision
  - b. False negatives costly: Focus on Recall

# **9. Tuning Hyperparameters And Stopping Criteria**



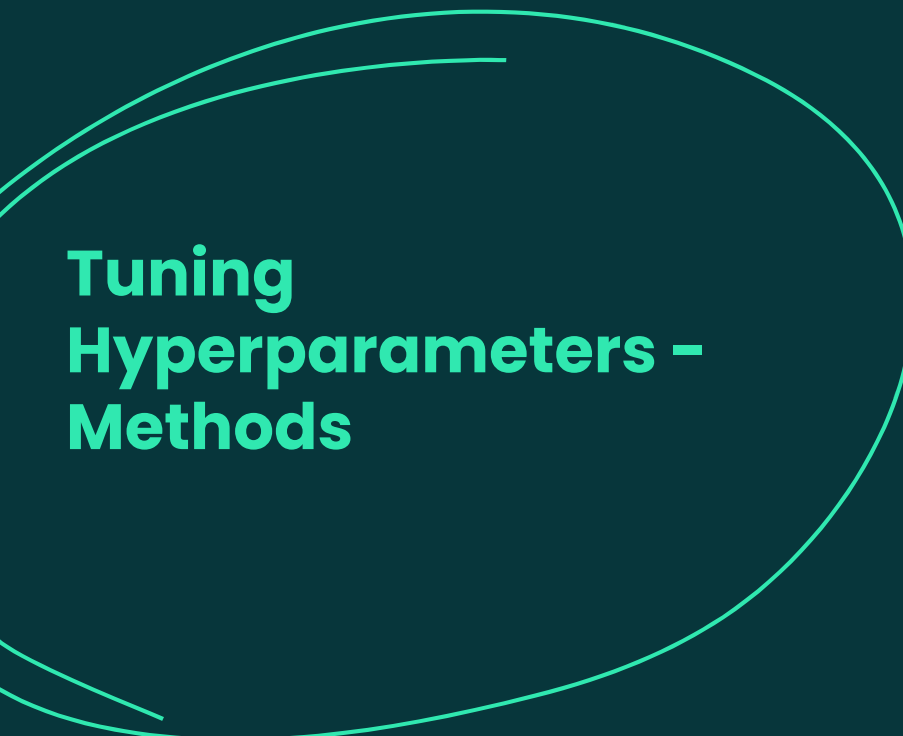
# Tuning Hyperparameters – Definition

What are Hyperparameters?

- Settings you choose before training a machine learning model

Examples

- Learning rate (how fast the model updates parameters)
- Number of trees, Max depth of a tree (in Random Forest / Boosting)
- Regularization strength (L1/L2)
- Number of epochs (in neural networks)

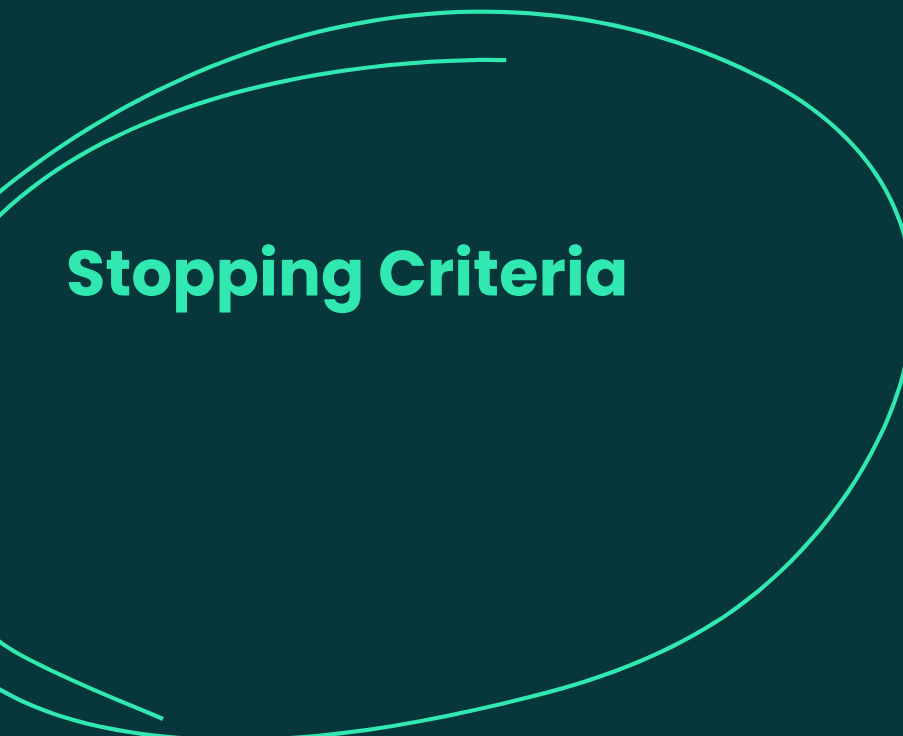


## Tuning Hyperparameters – Methods

1. Manual Search: Try values based on intuition and iterate
2. Grid Search: Try all combinations of predefined values
3. Random Search: Sample random combinations
4. Bayesian Optimization: choose next best parameters intelligently using Bayesian Models
  - a. Gaussian Processes
  - b. Tree-structured Parzen Estimators (TPE)
5. Gradient-Based Optimization

# Tuning Hyperparameters – Recommendations

| Approach              | Data/ Model            |
|-----------------------|------------------------|
| Manual Search         | Small Problems         |
| Grid Search           | Small search space     |
| Random Search         | High-dimensional space |
| Bayesian Optimization | Expensive models       |
| Gradient Based        | Neural Networks        |



## Stopping Criteria

1. Epoch-Based Stopping: Stop after a fixed number of iterations
2. Convergence-Based Stopping: Stop when the model stops improving meaningfully
  - a. Change in loss  $<$  tolerance ( $\epsilon$ )
  - b. Parameter updates become very small
3. Early Stopping: based on validation metric (loss, accuracy, etc)
4. Threshold Stopping: Stop once a target performance is reached

## Stopping Criteria - Model-wise Recommendations

| Criteria                 | Models/ Loss function       |
|--------------------------|-----------------------------|
| Epoch-based              | All Models                  |
| Convergence              | For Gradient Descent        |
| Early Stopping           | Neural Network,<br>Boosting |
| Performance<br>Threshold | Business Use case           |

# **10. Improving Robustness with Cross-validation**



## Improving Robustness

### (a) K-Fold Cross Validation

- Split into K folds (e.g., 5 or 10)
- Train on K-1, test on 1
- Rotate

### (b) Variants

1. Stratified K-Fold: Keeps class balance
2. Time Series CV: Rolling window
3. Repeated K-Fold: Multiple runs for stability



## Improving Robustness

### (c) Other robustness techniques

- Regularization (L1, L2)
- Early stopping (boosting, NN)
- Ensembling:
  - Bagging (Random Forest)
  - Boosting
  - Stacking



**Thank you!**